Aprendizaje por Refuerzo

Conceptos Fundamentales

Deep Q-Learning · Políticas · Recompensas · Entornos

¿Qué es el Aprendizaje por Refuerzo?

Un agente aprende a tomar decisiones interactuando con un entorno, maximizando una recompensa acumulada a lo largo del tiempo.

Aprendizaje activo

El agente toma acciones y observa sus consecuencias directamente en el entorno.

Prueba y error

No existe supervisión directa. El agente descubre qué funciona explorando.

Recompensa diferida

Las consecuencias de una acción pueden manifestarse mucho más tarde.

Elementos Fundamentales

Agente

El tomador de decisiones. Observa el estado, elige acciones y aprende de las recompensas.

Entorno

El mundo con el que interactúa el agente. Recibe acciones y devuelve nuevos estados y recompensas.

Estado (s)

Representación de la situación actual del entorno. Base sobre la que el agente toma decisiones.

Acción (a)

Decisión tomada por el agente en un estado dado. Puede ser discreta o continua.

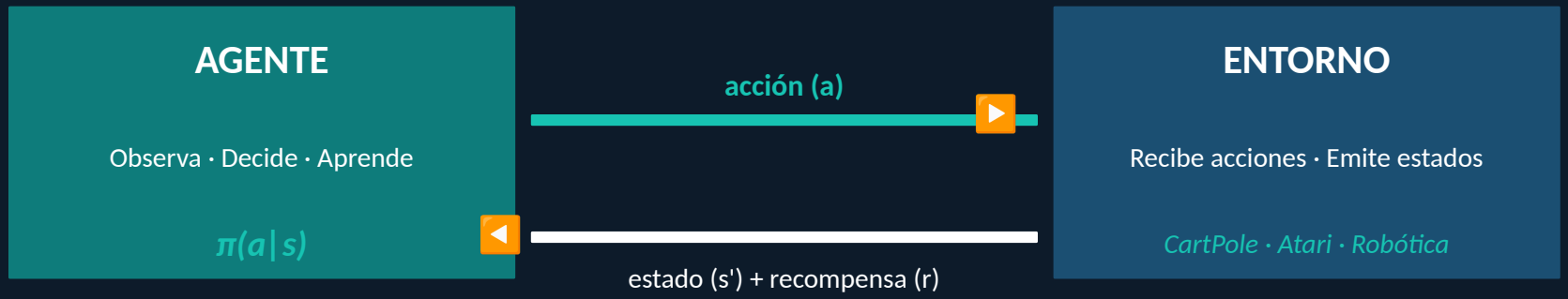
Recompensa (r)

Señal numérica que indica qué tan buena fue la acción tomada. El agente busca maximizarla.

Política (π)

Estrategia del agente: mapeo de estados a acciones. Es lo que el algoritmo entrena.

El Ciclo Agente-Entorno



Objetivo: maximizar $\Sigma\, \gamma^t \cdot r_t$

γ = factor de descuento · r_t = recompensa en paso t

Funciones de Valor

Función de Valor V(s)

$$V^{\pi}(s) = E[\Sigma \gamma^t r_t \mid s_0 = s, \pi]$$

¿Cuánta recompensa futura espero obtener si estoy en el estado s y sigo la política π?

Ejemplo:
En un juego de ajedrez, V(s) indica qué tan buena es la posición actual de las piezas.

Función Q(s, a)

$$Q^{\pi}(s,a) = E[\Sigma \gamma^t r_t \mid s_0=s, a_0=a, \pi]$$

¿Cuánta recompensa futura espero si tomo la acción a en el estado s y luego sigo π?

Clau DQN:
DQN aprende Q(s,a) directamente con una red neuronal profunda.

Deep Q-Network (DQN)

DQN combina Q-Learning con redes neuronales profundas para aprender en entornos de alta dimensión (ej. píxeles de Atari).

Red Neuronal

$Q(s,a;\theta)$

Aproxima la función Q. Recibe el estado como entrada y devuelve un valor Q por cada acción posible.

Replay Buffer

SequentialMemory

Almacena experiencias (s, a, r, s'). El agente aprende de muestras aleatorias, rompiendo correlaciones temporales.

Red Objetivo

Target Network

Copia estable de la red principal. Se actualiza periódicamente para dar estabilidad al entrenamiento.

Ecuación de Bellman: $Q(s,a) \leftarrow r + \gamma \cdot \max Q(s',a')$ — $\text{Loss} = [Q(s,a;\theta) - \text{target}]^2$

Políticas de Exploración

Dilema fundamental: Explorar (probar cosas nuevas) vs. Explotar (usar lo que ya funciona)

ε-Greedy (Epsilon-Greedy)

Usada en DQN · SB3

- Con probabilidad ε: acción aleatoria (exploración)
- Con probabilidad 1-ε: acción con mayor Q (explotación)
- ε se reduce gradualmente durante el entrenamiento
- Parámetros: exploration_initial_eps=1.0 → exploration_final_eps=0.05

Boltzmann (Softmax)

Usada en keras-rl

- Selecciona acciones según una distribución de probabilidad
- $P(a|s) \propto \exp(Q(s,a) / \tau)$ — τ es la temperatura
- τ alta → más exploración · τ baja → más explotación
- Tiene en cuenta los valores Q relativos de todas las acciones

Tipos de Algoritmos de RL

Q-Learning / DQN

Off-Policy · Basado en Valor

Aprende la función Q. DQN extiende Q-Learning con redes neuronales para espacios grandes.

SARSA

On-Policy · Basado en Valor

Similar a Q-Learning pero aprende de la política actual, no de la óptima.

Policy Gradient (REINFORCE)

On-Policy · Basado en Política

Optimiza directamente la política mediante gradiente ascendente sobre la recompensa esperada.

Actor-Critic (A2C / A3C)

On/Off-Policy · Híbrido

Combina basado en valor (critic) y basado en política (actor). Más estable y eficiente.

PPO

On-Policy · Estado del Arte

Proximal Policy Optimization. Muy popular por su robustez. Recomendado como punto de partida.

SAC

Off-Policy · Continuo

Soft Actor-Critic. Ideal para espacios de acción continuos (robótica, control).

Herramientas y Librerías Modernas

Gymnasium

Entornos

Sucesor de OpenAI Gym. Proporciona cientos de entornos estandarizados: CartPole, Atari, MuJoCo, etc.

Stable-Baselines3

Algoritmos

Implementaciones fiables de DQN, PPO, SAC, A2C, TD3. Compatible con PyTorch. Activamente mantenido.

PyTorch

Deep Learning

Backend principal de SB3. Flexible, dinámico y con gran ecosistema de herramientas de RL.

TensorBoard

Visualización

Monitoriza el entrenamiento en tiempo real: recompensas, pérdidas, exploración epsilon, etc.

```
pip install gymnasium stable-baselines3[extra] torch  ·  from stable_baselines3 import DQN, PPO, SAC
```

Resumen

1. El agente aprende interactuando con el entorno mediante recompensas
2. Las funciones V(s) y Q(s,a) estiman la recompensa futura esperada
3. DQN aproxima Q(s,a) con una red neuronal profunda
4. El Replay Buffer y la Target Network dan estabilidad al entrenamiento
5. La exploración (ε-greedy) equilibra descubrir y explotar conocimiento
6. Gymnasium + Stable-Baselines3 es el stack moderno recomendado